

Guía de Estudio — Tema 04

ORM avanzado + puente a interfaz MVC

Materia: Laboratorio de Programación y Lenguajes · IF009

Institución: Universidad Nacional de Tierra del Fuego — Instituto IDEI

Ciclo lectivo: 2026 · Semana 8

Autora: Dra. Sofía (study-guide-writer) · Generado: 2026-04-29

"Si un alumno puede estudiarlo solo, lo hicimos bien."

Esta guía es para **vos**, el estudiante. No reemplaza la clase — la profundiza para que puedas estudiar de forma autónoma, repasar antes del parcial y practicar sin depender del entorno de la cátedra.

0. Antes de empezar: qué sabés y qué vas a aprender

Lo que ya sabés (requisito de esta guía)

- `filter()`, `get()`, `order_by()`, `all()` sobre modelos Django
- `.save()`, `.delete()` individual
- Managers personalizados con `get_queryset()` y métodos de instancia
- ORM básico sobre el dominio Biblioteca (Autor, Libro, Lector)

Si alguno de estos puntos te genera dudas, revisá la **práctica anterior (orm.pdf)** antes de continuar.

Lo que vas a aprender en este tema

Al terminar esta guía podrás:

1. Explicar qué es la *lazy evaluation* de un `QuerySet` y cuándo se ejecuta la SQL.
 2. Construir consultas con `Q objects` (OR, AND, NOT) y `F expressions` (operaciones sobre campos).
 3. Usar `aggregate()` para estadísticas globales y `annotate()` para enriquecer cada objeto.
 4. Detectar y resolver el problema N+1 con `select_related` y `prefetch_related`.
 5. Implementar una vista con clase base `View`, conectarla en `urls.py` y pasarle contexto a un template.
 6. Escribir templates Django completos usando las 4 construcciones de DTL: variables, filtros, tags y comentarios.
 7. Aplicar herencia de templates con `{% extends %}`, `{% block %}` y `{% include %}`.
-

1. Introducción: por qué este tema importa

Este tema es el **punto de conexión entre los datos y la interfaz** de tu aplicación Django. Hasta ahora trabajaste con modelos en el shell — leer, crear, modificar datos. Eso es el 50% de una aplicación web.

El otro 50% es hacer que esos datos lleguen al navegador del usuario de forma correcta, eficiente y segura.

En este tema vas a aprender:

- **Cómo Django decide cuándo ir a la base de datos** (lazy evaluation) — vital para no hacer consultas innecesarias.
- **Cómo construir consultas complejas** que `filter()` solo no puede expresar.
- **Cómo medir y optimizar el rendimiento** de tus queries antes de que se conviertan en un problema.
- **Cómo conectar tus modelos a una vista y un template** — la cadena completa URL → View → Model → Template.

El dominio de práctica es **BlogApp** — `Post`, `Category`, `Comment` — que usaremos también en Semana 9.

2. Objetivos de aprendizaje

Al terminar esta guía, serás capaz de:

#	Objetivo	Nivel Bloom
1	Describir la lazy evaluation de QuerySets y sus puntos de evaluación	Comprender (2)
2	Construir consultas con Q objects, F expressions, annotate y aggregate	Aplicar (3)
3	Diagnosticar el problema N+1 con <code>connection.queries</code> y resolverlo	Analizar (4)
4	Implementar una CBV con <code>view</code> base y conectarla a <code>urls.py</code>	Aplicar (3)
5	Escribir templates DTL con herencia, filtros, forloop, with e include	Aplicar (3)
6	Integrar la cadena completa ORM → Vista → Template → Navegador	Sintetizar (5)

3. Conceptos previos necesarios

Antes de continuar, verificá que podés hacer esto sin ayuda:

- ☐ [] Crear un modelo Django con campos básicos y relaciones FK/M2M
- ☐ [] Ejecutar `python manage.py shell` y usar `filter()`, `get()`, `order_by()`
- ☐ [] Crear un Manager con `get_queryset()` que filtre objetos
- ☐ [] Entender qué es un `request` y un `response` HTTP a nivel conceptual

Si marcaste todos → estás listo/a. Si alguno falla → revisá la práctica anterior primero.

4. Desarrollo teórico

4.1 QuerySet API avanzado

Ver Filminas 2 a 12 en la clase teórica

4.1.1 ¿Qué es un QuerySet?

Un `QuerySet` es un **objeto Python que representa una consulta a la base de datos**. La clave — y lo que lo hace diferente a otras ORMs — es que es **diferido** (lazy): no ejecuta la consulta SQL hasta que realmente necesita los datos.

```
# Esta línea NO ejecuta SQL:
qs = Post.objects.filter(published=True).order_by("-created_at")
# qs es un objeto que *describe* la consulta — todavía no habló con la BD

# Esta línea SÍ ejecuta SQL (la iteración fuerza la evaluación):
for post in qs:
    print(post.title)
```

La **caché interna** es el otro aspecto importante: una vez que el `QuerySet` se evalúa, el resultado queda guardado en el objeto. La segunda iteración del mismo `qs` no va a la base de datos.

¿Por qué importa esto? En Django, el mismo `QuerySet` puede aparecer varias veces en el código (una en la vista, otra en el template). Si se evalúa una vez, la segunda es gratuita. Pero si modificás el `QuerySet` entre medio, la caché se invalida.

4.1.2 Los 6 puntos de evaluación

El `QuerySet` se evalúa (ejecuta la SQL) en estas situaciones:

Situación	Ejemplo
Iteración	<code>for post in qs:</code>
<code>list()</code>	<code>list(qs)</code>
Slicing con pasos	<code>qs[0:5]</code>
<code>bool()</code> o <code>if</code>	<code>if qs:</code>
<code>len()</code>	<code>len(qs)</code>
<code>repr()</code> (shell)	escribir <code>qs</code> en el shell

Consejo práctico: si solo querés saber si hay resultados, usá `qs.exists()` en lugar de `if qs: .exists()` genera un `SELECT 1` muy eficiente en lugar de traer todos los objetos.

4.1.3 Métodos nuevos del QuerySet

Estos métodos amplían lo que ya sabés de `filter()` y `get()`:

Métodos de recuperación:

```
# first() y last() → objeto o None (no lanza DoesNotExist)
post = Post.objects.filter(published=True).first()

# exists() → bool eficiente
hay_posts = Post.objects.filter(published=True).exists()

# count() → SELECT COUNT(*) sin traer objetos
cantidad = Post.objects.filter(published=True).count()
```

Métodos de eficiencia:

```
# values() → lista de dicts (sin instanciar modelos)
titulos = Post.objects.values("title", "author__username")
# [{"title": "...", "author__username": "..."}, ...]

# values_list() → lista de tuplas o valores
ids = Post.objects.values_list("id", flat=True)
# QuerySet[1, 2, 3]

# only() → instancias parciales (solo los campos pedidos)
posts = Post.objects.only("title", "created_at")
# Si accedés a un campo no incluido → SQL extra automático

# defer() → todo menos los campos pesados
posts = Post.objects.defer("body", "raw_content")
```

Operaciones atómicas:

```
# get_or_create → busca o crea, devuelve (obj, creado: bool)
cat, created = Category.objects.get_or_create(
    slug="python",
    defaults={"name": "Python"}
)

# update() → SQL UPDATE directo, no llama .save()
Post.objects.filter(author=user).update(published=True)

# bulk_create → inserts masivos sin save() individual
Post.objects.bulk_create([
    Post(title="Post A", author=user),
    Post(title="Post B", author=user),
])
```

Importante: `update()` y `bulk_create()` no disparan signals de Django (`post_save`, `pre_save`). Si tenés lógica en esos signals, necesitás iterar con `.save()` individual.

4.2 Consultas dinámicas: Q objects y F expressions

Ver Filminas 13 a 23

4.2.1 Q objects: condiciones lógicas complejas

`filter()` solo puede expresar condiciones AND implícitas. Para OR, NOT, o combinaciones dinámicas, necesitas `Q`:

```
from django.db.models import Q

# OR: publicados O del usuario actual
Post.objects.filter(Q(published=True) | Q(author=request.user))
# SQL: WHERE published = true OR author_id = 1

# NOT: no staff
Post.objects.filter(~Q(author__is_staff=True))

# Combinación dinámica – el uso más poderoso:
filters = Q() # Q vacío: elemento neutro
if search:
    filters &= Q(title__icontains=search)
if category_id:
    filters &= Q(category_id=category_id)
Post.objects.filter(filters, published=True)
```

Operadores de Q:

Operador	Significado
	OR lógico
&	AND lógico
~	NOT lógico

4.2.2 F expressions: operar en SQL

`F` permite referenciar el valor de un campo de la BD en la query, sin traerlo a Python:

```
from django.db.models import F

# Incrementar un contador SIN traer el objeto – atómico
Post.objects.filter(pk=pk).update(views=F("views") + 1)
# SQL: UPDATE blog_post SET views = views + 1 WHERE id = pk

# Comparar dos campos de la misma fila
Post.objects.filter(updated_at__gt=F("created_at"))
```

Por qué `F` es importante: si haces `post.views += 1; post.save()`, hay un *race condition* — si dos usuarios acceden al mismo post en el mismo instante, ambos leen `views = 5`, ambos guardan `views = 6`. El incremento correcto sería `7`. Con `F`, el incremento ocurre en SQL, de forma atómica.

4.3 Estadísticas: `aggregate()` y `annotate()`

Ver Filminas 17 a 19

4.3.1 aggregate(): estadísticas globales

```
from django.db.models import Count, Avg, Sum, Max, Min

# Un dict con el resultado global del QuerySet
stats = Post.objects.aggregate(
    total=Count("id"),
    avg_views=Avg("views"),
    max_views=Max("views")
)
# Resultado: {"total": 42, "avg_views": 287.3, "max_views": 1500}
```

4.3.2 annotate(): campo calculado por objeto

```
from django.db.models import Count

# Cada Category ahora tiene un atributo .post_count
categories = Category.objects.annotate(
    post_count=Count("post")
).order_by("-post_count")

for cat in categories:
    print(f"{cat.name}: {cat.post_count} posts")
```

La diferencia clave:

	aggregate()	annotate()
Resultado	Un dict	QuerySet con campo extra
SQL	SELECT COUNT(*) FROM ...	GROUP BY
Cuándo usar	"¿Cuántos posts tiene el blog?"	"¿Cuántos posts tiene <i>cada</i> categoría?"

4.4 Performance: el problema N+1

Ver Filminas 20 a 23

4.4.1 El bug silencioso

El problema N+1 es el bug de performance más común en Django y el más silencioso — no da error, solo hace lenta la aplicación:

```
# ❌ N+1: 1 query para posts + 1 por post para su autor
posts = Post.objects.all() # query 1
for post in posts:
    print(post.author.username) # query 2, 3, 4... N+1
```

Con 50 posts y 3 comentarios por post: fácilmente 200+ queries por request.

4.4.2 Diagnóstico con connection.queries

```
from django.db import connection, reset_queries
from django.conf import settings

settings.DEBUG = True    # solo en desarrollo
reset_queries()

# tu código aquí...

print(f"Queries ejecutadas: {len(connection.queries)}")
# Para ver el SQL: [q['sql'] for q in connection.queries]
```

4.4.3 Solución: select_related y prefetch_related

```
# select_related → JOIN SQL (para FK y OneToOne)
# Una sola query
posts = Post.objects.select_related("author").all()

# prefetch_related → 2 queries con IN (para M2M y reverse FK)
posts = Post.objects.prefetch_related("categories", "comments").all()

# Combinación real en una vista de listado
posts = Post.objects.select_related("author")\
    .prefetch_related("categories")\
    .filter(published=True)\
    .order_by("-created_at")
```

Regla práctica:

- FK y OneToOne → `select_related`
- ManyToMany y reverse FK → `prefetch_related`
- Podés combinar ambos en el mismo QuerySet

4.5 Managers personalizados en BlogApp

Ver Filmina 11

```
# blog/models.py
class PublishedManager(models.Manager):
    def get_queryset(self):
        return super().get_queryset().filter(published=True)

    def recientes(self, n=10):
        """Top N publicados, eficiente."""
        return self.get_queryset()\
            .select_related("author")\
            .only("title", "created_at", "author__username")\
            .order_by("-created_at")[:n]

class Post(models.Model):
    objects = models.Manager()    # siempre declarar el default
    published = PublishedManager() # custom manager
```

El Manager encapsula la estrategia de consulta — incluyendo el `select_related` y el `only()`. Cualquier código que use `Post.published.recientes()` automáticamente recibe datos optimizados.

4.6 Vistas con clase base: Django MVT

Ver Filminas 24 a 30

4.6.1 El ciclo request/response

```
Browser → HTTP Request
  → urls.py      (enrutador: ¿qué vista maneja esta URL?)
  → View (clase)  (controlador: pide datos, pasa al template)
  → Model        (datos: QuerySet, instancias)
  → Template     (presentación: HTML con datos)
← HTTP Response ← HTML generado
```

Responsabilidades:

- **Model:** datos y lógica de dominio. No conoce la request.
- **Template:** presentación HTML. Sin lógica de negocio.
- **View:** orquesta. Recibe request, consulta modelos, pasa contexto al template, devuelve response.

4.6.2 View como clase base

```
from django.views import View
from django.shortcuts import render
from .models import Post

class PostListView(View):
    template_name = "blog/post_list.html"

    def get(self, request):
        posts = Post.objects.select_related("author")\
            .filter(published=True)\
            .order_by("-created_at")
        return render(request, self.template_name, {"posts": posts})
```

Puntos clave:

- `as_view()` convierte la clase en callable para `urls.py`
- `dispatch()` lee `request.method` y llama `get()`, `post()`, etc.
- Si el método HTTP no tiene método en la clase → 405 Method Not Allowed
- Cada request crea una instancia nueva — thread-safe

4.6.3 Conectar en `urls.py`

```
# blog/urls.py
from django.urls import path
from .views import PostListView, PostDetailView

app_name = "blog" # namespace para {% url 'blog:post-list' %}
urlpatterns = [
    path("", PostListView.as_view(), name="post-list"),
    path("<int:pk>/", PostDetailView.as_view(), name="post-detail"),
]
```

4.7 Django Template Language (DTL) completo

Ver Filminas 31 a 47

4.7.1 Los 4 constructos fundamentales

Constructo	Sintaxis	Propósito
Variable	<code>{{ variable }}</code>	Mostrar un valor del contexto
Filtro	<code>{{ valor filtro }}</code>	Transformar al mostrar
Tag	<code>{% tag %}</code>	Lógica: bucles, condicionales, herencia
Comentario	<code>{# texto #}</code>	Documentación — no se renderiza

4.7.2 Variables y notación de punto

```
# La vista pasa el contexto como dict:
context = {"post": post_obj, "posts": queryset}
```

```
{{ post.title }}           <!-- atributo title -->
{{ post.author.username }} <!-- encadenado -->
{{ posts.0.title }}        <!-- primer elemento -->
```

Django resuelve el punto probando: **atributo** → **clave dict** → **índice lista** → **método callable**.

Seguridad: los atributos que comienzan con `_` son inaccesibles por defecto.

Auto-escape: Django escapa HTML automáticamente. Si escribís `{{ user_input }}`, los caracteres `<`, `>`, `&` se convierten en `<`, `>`, `&`. Esto previene ataques XSS. Para desactivar (solo con contenido de confianza): `{{ valor|safe }}`.

4.7.3 Filtros más usados

```
{{ post.title|lower }}           <!-- minúsculas -->
{{ post.created_at|date:"d/m/Y" }} <!-- 28/06/2025 -->
{{ post.body|truncatewords:30 }} <!-- primeras 30 palabras + ... -->
{{ post.body|linebreaks }}        <!-- \n → <p> / <br> -->
{{ post.subtitle|default:"Sin subtítulo" }} <!-- fallback si vacío -->
{{ comments|length }}            <!-- cantidad de elementos -->
{{ texto|lower|truncatewords:30 }} <!-- encadenados: izq a der -->
```

4.7.4 Tags de control: for e if

`{% for %}` :

```
{% for post in posts %}
  <p>{{ forloop.counter }}. {{ post.title }}</p>
{% empty %}
  <p>No hay posts.</p>
{% endfor %}
```

Variables de `forloop` :

Variable	Descripción
<code>forloop.counter</code>	Índice desde 1
<code>forloop.counter0</code>	Índice desde 0
<code>forloop.first</code>	<code>True</code> en la primera iteración
<code>forloop.last</code>	<code>True</code> en la última iteración
<code>forloop.parentloop</code>	Loop externo (en loops anidados)

`{% if %}` :

```
{% if posts %}
    <p>Hay {{ posts|length }} publicaciones.</p>
{% elif drafts %}
    <p>Solo hay borradores.</p>
{% else %}
    <p>Sin contenido.</p>
{% endif %}
```

⚠ **Trampa de precedencia:** en DTL, `and` y `or` se evalúan de izquierda a derecha, no como en Python donde `and` tiene precedencia. Para lógica compleja, anidar `{% if %}` .

4.7.5 `{% with %}`: alias de variables

```
{% with author=post.author %}
    <p>{{ author.get_full_name }}</p>
    <p>{{ author.email }}</p>
{% endwith %}
```

Útil para evitar lookups FK repetidos dentro de un bloque.

4.7.6 Archivos estáticos: `{% load static %}`

```
{% load static %} <!-- debe ir en cada template que use {% static %} -->

<link rel="stylesheet" href="{% static 'blog/css/styles.css' %}">

```

En `settings.py` :

```
STATIC_URL = "/static/"
STATICFILES_DIRS = [BASE_DIR / "static"]
```

4.7.7 URLs por nombre: `{% url %}`

```
<!-- Nunca hardcodear /blog/ -->
<a href="{% url 'blog:post-list' %}">Inicio</a>
<a href="{% url 'blog:post-detail' pk=post.pk %}">Ver</a>
```

Si cambiás la URL en `urls.py`, todos los templates siguen funcionando.

4.7.8 Comentarios DTL

```
{# Comentario de una línea — no se renderiza #}

{% comment "Sección pendiente" %}
    <!-- Todo este bloque es ignorado -->
{% endcomment %}
```

A diferencia de `<!-- -->` HTML, los comentarios DTL **no llegan al navegador**.

4.7.9 Herencia de templates

El problema sin herencia: cada template repite el mismo `<head>`, navbar, footer.

La solución: `{% extends %}` + `{% block %}`:

```
<!-- base.html — el esqueleto -->
<!DOCTYPE html>
<html>
<head>
    <title>{% block title %}BlogApp{% endblock %}</title>
</head>
<body>
    <nav>...</nav>
    <main>{% block content %}{% endblock %}</main>
</body>
</html>
```

```
<!-- post_list.html — el hijo -->
{% extends "blog/base.html" %}    <!-- DEBE ser la primera línea -->

{% block title %}Listado{% endblock %}

{% block content %}
    <!-- solo el contenido específico de esta página -->
{% endblock %}
```

Reglas importantes:

1. `{% extends %}` debe ser **la primera línea** del template hijo.
2. Solo el contenido dentro de `{% block %}` puede sobrescribirse.
3. `{{ block.super }}` incluye el contenido del bloque padre y agrega el propio — sin perder nada.

4.7.10 Partial: `{% include %}`

```
<!-- post_card.html — componente reutilizable -->
<article class="post-card">
    <h2>{{ post.title }}</h2>
    <p>{{ post.body|truncatewords:25 }}</p>
</article>
```

```
<!-- post_list.html — usa el partial -->
{% for post in posts %}
    {% include "blog/partials/post_card.html" with post=post %}
{% endfor %}
```

Ventaja: cualquier otra página que muestre tarjetas de post usa el mismo partial — un solo lugar para cambiar el diseño.

5. Ejemplos trabajados

Ejemplo 1 — QuerySet con lazy evaluation y diagnóstico

Situación: tenés una vista de listado que muestra posts con el nombre del autor. Querés asegurarte de que no hay N+1.

Paso 1: identificar el problema

```
# views.py — versión con N+1
def get(self, request):
    posts = Post.objects.filter(published=True)
    # Al iterar en el template: post.author.username → 1 query por post
    return render(request, self.template_name, {"posts": posts})
```

Paso 2: diagnosticar en el shell

```
from django.db import connection, reset_queries
from django.conf import settings

settings.DEBUG = True
reset_queries()

posts = list(Post.objects.filter(published=True))
for p in posts:
    _ = p.author.username

print(f"Queries: {len(connection.queries)}") # → N+1
```

Paso 3: resolver

```
# views.py — versión optimizada
def get(self, request):
    posts = Post.objects.select_related("author")\
        .filter(published=True)\
        .order_by("-created_at")
    return render(request, self.template_name, {"posts": posts})
```

Verificación: mismo diagnóstico → ahora muestra 1 query .

Ejemplo 2 — Búsqueda dinámica con Q objects

Situación: una barra de búsqueda que filtra por título, categoría o ambos. Si no se provee ninguno, muestra todos los publicados.

```
# views.py
class PostSearchView(View):
    template_name = "blog/post_search.html"

    def get(self, request):
        search = request.GET.get("q", "").strip()
        category_slug = request.GET.get("cat", "").strip()

        filters = Q(published=True) # base: siempre publicados

        if search:
            filters &= (
                Q(title__icontains=search) |
                Q(body__icontains=search)
            )

        if category_slug:
            filters &= Q(categories__slug=category_slug)

        posts = Post.objects.select_related("author")\
            .filter(filters)\
            .distinct()\
            .order_by("-created_at")

        return render(request, self.template_name, {
            "posts": posts,
            "search": search,
            "category_slug": category_slug,
        })
```

Notas:

- `distinct()` es necesario cuando hay joins con M2M (`categories`) para evitar duplicados.
 - Los `Q` se construyen solo si el usuario proveyó el parámetro — eficiente.
-

Ejemplo 3 — Template completo con herencia y forloop

```
{% extends "blog/base.html" %}
{% load static %}

{% block title %}Búsqueda: {{ search }}{% endblock %}

{% block extra_head %}
    {{ block.super }}
    <link rel="stylesheet" href="{% static 'blog/css/search.css' %}">
{% endblock %}

{% block content %}
    <h1>Resultados para "{{ search }}"</h1>

    {% if posts %}
        <p>{{ posts|length }} resultado{{ posts|length|pluralize }} encontrado{{ posts|length|pluralize }}.</p>

        {% for post in posts %}
            <article {% if forloop.first %}class="first-result"{% endif %}>
                {% with author=post.author %}
                    <h2>
                        <a href="{% url 'blog:post-detail' pk=post.pk %}">
                            {{ forloop.counter }}. {{ post.title }}
                        </a>
                    </h2>
                    <p class="meta">
                        Por {{ author.get_full_name|default:author.username }}
                        – {{ post.created_at|date:"d/m/Y" }}
                    </p>
                    {% endwith %}
                    <p>{{ post.body|truncatewords:30 }}</p>
                </article>

                {% if forloop.last %}
                    <p><em>Fin – {{ forloop.counter }} resultado{{ forloop.counter|pluralize }}.</em></p>
                {% endif %}
            {% endfor %}
        {% else %}
            <p>No se encontraron resultados para "{{ search|escape }}"</p>
            <a href="{% url 'blog:post-list' %}">Ver todos los posts</a>
        {% endif %}
    {% endblock %}
```

6. Puntos clave y resumen

ORM avanzado

- **Lazy evaluation:** el QuerySet no consulta la BD hasta que se consume (iteración, `list()`, `bool()`, etc.)
- **Caché interna:** una vez evaluado, la segunda iteración no va a la BD
- **`exists()` vs `if qs:`** : `exists()` es más eficiente cuando solo importa si hay resultados
- **`only()` / `defer()`** : traer solo los campos necesarios reduce el tiempo de transferencia
- **`get_or_create()`** : devuelve (objeto, `fue_creado_bool`) — atómico a nivel Django
- **`update()` / `bulk_create()`** : no llaman `.save()`, no disparan signals — más rápidos para operaciones masivas

Consultas dinámicas

- **Q objects:** permite OR (`|`), AND (`&`), NOT (`~`) en filtros; se pueden construir dinámicamente
- **F expressions:** referencia a columnas SQL; permite operaciones atómicas sin traer datos a Python
- `aggregate()` : estadística global → devuelve `dict`
- `annotate()` : campo calculado por objeto → devuelve QuerySet enriquecido

Performance N+1

- **El problema:** acceder a FK dentro de un loop genera 1 query por iteración
- `select_related` : JOIN SQL — para FK y OneToOne — 1 sola query
- `prefetch_related` : 2 queries con IN — para M2M y reverse FK
- **Diagnóstico:** `settings.DEBUG = True` + `len(connection.queries)`

CBV con View base

- `as_view()` : convierte la clase en callable para `urls.py`
- `dispatch()` : enruta el request al método correcto (`get()` , `post()` , etc.)
- `get_object_or_404()` : 404 automático si el objeto no existe
- `app_name` en `urls.py` : habilita el namespace `'blog:post-list'`

DTL

- **4 constructos:** variable `{{ }}` , filtro `|` , tag `{% %}` , comentario `{# #}`
- **Auto-escape:** protección XSS incorporada — `{{ var }}` escapa HTML
- `{% for %}` con `{% empty %}` y variables `forloop.*`
- `{% if %}` : precaución con precedencia — DTL es izquierda a derecha
- `{% with %}` : alias para evitar lookups repetidos
- `{% load %}` : no se hereda — cada template que usa una librería debe cargarlo
- `{% extends %}` : primera línea del template hijo — herencia de estructura
- `{{ block.super }}` : incluir el contenido del padre y agregar el propio
- `{% include %}` : partials reutilizables — la tarjeta de post como componente

7. Autoevaluación

Respondé estas preguntas para verificar tu comprensión. Las respuestas no se dan aquí — si no podés responder, volvé a la sección correspondiente.

1. ¿Qué hace este código? ¿Cuántas queries SQL ejecuta?

```
qs = Post.objects.filter(published=True)
if qs:
    print(qs.count())
```

2. Escribí el código para obtener los 5 posts con más comentarios (usando `annotate` y `Count`).
3. ¿Cuál es la diferencia entre `select_related` y `prefetch_related` ? ¿Cuándo usarías cada uno?
4. Tenés el siguiente template. ¿Qué está mal?

```
<p>Hola</p>
{% extends "base.html" %}
{% block content %}...{% endblock %}
```

5. ¿Cómo mostrarías la fecha de creación de un post en formato `"29 de abril de 2026"` usando un filtro de DTL?
6. ¿Qué hace `{{ block.super }}` ? ¿Cuándo lo usarías?
7. Escribí una vista CBV que muestre solo los posts de una categoría dada (recibida como `slug` en la URL), usando `get_object_or_404` para la categoría.
8. ¿Qué problema tiene este template?

```
{% for post in posts %}
    {% if post.published and post.author == user or post.featured %}
        {# mostrar #}
    {% endif %}
{% endfor %}
```

8. Glosario

Término	Definición
QuerySet	Objeto Python que representa una consulta a la BD — diferido (lazy), cacheable
Lazy evaluation	Estrategia de evaluación donde el cómputo se pospone hasta que el resultado es necesario
N+1 problem	Bug de performance donde se ejecutan N queries adicionales (una por cada FK en un loop) después de la query inicial
select_related	Optimización ORM que usa JOIN SQL para resolver FKs en una sola query
prefetch_related	Optimización ORM que usa queries separadas con IN para resolver M2M y reverse FK
Q object	Objeto de Django que encapsula una condición de consulta, combinable con OR/AND/NOT
F expression	Referencia a un campo de la BD usable en queries SQL sin traer el valor a Python
aggregate()	Función de QuerySet que devuelve estadísticas globales como un dict
annotate()	Función de QuerySet que agrega campos calculados a cada objeto del resultado
CBV	Class-Based View — vista implementada como clase Python con métodos por verbo HTTP
dispatch()	Método de View que enruta el request al método de clase correcto según el método HTTP
as_view()	Método de clase que convierte la CBV en un callable aceptado por urls.py
MVT	Model-View-Template — la arquitectura de Django (análoga a MVC)
DTL	Django Template Language — lenguaje de templates de Django
Herencia de templates	Mecanismo de Django para compartir estructura HTML base entre múltiples templates
Partial	Fragment de template reutilizable insertado con <code>{% include %}</code>
Auto-escape	Comportamiento de Django de escapar HTML en variables para prevenir XSS
XSS	Cross-Site Scripting — ataque que inyecta scripts maliciosos en páginas web
Race condition	Bug de concurrencia donde el resultado depende del orden de ejecución de operaciones paralelas

9. Referencias y lecturas recomendadas

Documentación oficial (lectura obligatoria)

- Django 6.0 — Making queries: docs.djangoproject.com/en/6.0/topics/db/queries/
- Django 6.0 — QuerySet API reference: docs.djangoproject.com/en/6.0/ref/models/querysets/
- Django 6.0 — Class-based views: docs.djangoproject.com/en/6.0/topics/class-based-views/
- Django 6.0 — The Django template language: docs.djangoproject.com/en/6.0/ref/templates/language/
- Django 6.0 — Built-in template tags and filters: docs.djangoproject.com/en/6.0/ref/templates/builtins/

Material adicional recomendado

- **Two Scoops of Django 3.x** — Daniel Roy Greenfeld & Audrey Roy Greenfeld — Cap. 6 (Model Best Practices) y Cap. 10 (CBVs)
- **Django for Professionals** — William S. Vincent — Cap. sobre ORM y Templates

Material fuente del curso

- `material/orm.pdf` — práctica anterior: ORM básico sobre dominio Biblioteca (16 páginas)
- `salida/cursadas/2026/plan-actualizado.md` — plan de la materia con contexto de esta clase